Práctica 2: Problema de los Lectores y Escritores Distribuido

Sistemas Distribuidos - Universidad de Zaragoza

Enrique José Guarás Lacasta 870709
Anastasia Shlyk 931807

22 de octubre de 2026

Resumen

Esta memoria describe el diseño y la implementación en Go de una solución distribuida al problema de los lectores y escritores, basada en el algoritmo de Ricart-Agrawala generalizado mediante una matriz de exclusión de operaciones. Se detalla la arquitectura de componentes, la máquina de estados del protocolo, una red de Petri que modela el comportamiento completo de un nodo, los diagramas de secuencia de los escenarios representativos, las decisiones de implementación y la batería de pruebas empleada para validar la corrección.

Índice

1. Introducción **2**

2. Arquitectura del sistema **2**
2.1. Visión global . . . 2
2.2. Estructura interna de un nodo: tres flujos concurrentes . . . 3
2.3. Gestor de ficheros y propagación de escrituras . . . 3

3. Diseño del algoritmo de Ricart-Agrawala generalizado **4**
3.1. De la versión clásica a la generalizada . . . 4
3.2. Matriz de exclusión . . . 4
3.3. Máquina de estados de un nodo . . . 4
3.4. Modelo formal: red de Petri del comportamiento de un nodo . . . 5
3.5. Diagramas de secuencia . . . 7
3.5.1. Conflicto entre dos escritores (postergación y desempate) . . . 8
3.5.2. Escritor frente a lector en sección crítica (propagación de la escritura) . . . 8
3.5.3. Lecturas concurrentes (sin postergación) . . . 8

4. Implementación en Go **11**
4.1. Tipos de mensajes y serialización . . . 11
4.2. Estado del nodo . . . 12
4.3. Preprotocolo (instrucciones (1)–(6)) . . . 13
4.4. Recepción de mensajes (instrucciones (10)–(15)) . . . 13
4.5. Postprotocolo (instrucciones (7)–(9)) . . . 14
4.6. Condiciones de carrera consideradas . . . 14

5. Pruebas y validación **15**
5.1. Estrategia . . . 15
5.2. Batería de pruebas . . . 15
5.3. Criterios de corrección comprobados . . . 15

6. Limitaciones conocidas y trabajo futuro **15**

7. Conclusión **15**

1. Introducción

El objetivo de la práctica es implementar el problema de los lectores y escritores en un entorno distribuido sin proceso coordinador, donde los N procesos participantes (lectores y escritores) tienen todos la misma responsabilidad en el sistema y deciden en función exclusivamente de su información local y de los mensajes que han recibido. Cada proceso mantiene una copia local de un fichero de texto, las lecturas trabajan sobre la copia propia y pueden producirse simultáneamente en varios nodos, mientras que las escrituras se realizan en exclusión mutua total y se propagan al resto de copias, de modo que tras cualquier escritura el contenido de todos los ficheros es idéntico.

La exclusión mutua distribuida clásica (Ricart-Agrawala) trata todas las entradas a sección crítica como conflictivas entre sí. Eso es suficiente para un mutex puro, pero desperdicia el paralelismo real del problema, donde dos lecturas no se excluyen. La generalización pedida por el guion consiste en parametrizar el algoritmo con una *matriz de exclusión* que define qué pares de operaciones son incompatibles, de modo que el mismo motor de sincronización resuelve tanto el mutex clásico como lectores/escritores (como cualquier problema con varios tipos de operación al que se quiera adaptar).

El resto del documento se centra en cómo hemos diseñado e implementado la solución, referenciando la arquitectura de cada nodo (Sección 2), el diseño del algoritmo generalizado con su máquina de estados, su red de Petri y sus diagramas de secuencia (Sección 3), los detalles de implementación en Go y las condiciones de carrera resueltas (Sección 4), la validación experimental (Sección **??**) y las limitaciones conocidas (Sección 6).

2. Arquitectura del sistema

2.1. Visión global

El sistema se compone de N procesos independientes (sin memoria compartida ni coordinador), identificados por un entero $pid \in \{1, \ldots, N\}$. La correspondencia entre *pid* y dirección de red (IP:puerto) se resuelve a partir de un fichero `users.txt` común otorgado, enmascarando la ubicación física de los procesos y permitiendo mover el despliegue del clúster de Raspberry Pi (ARM64) del laboratorio a cualquier otra máquina.

Cada nodo alberga tres componentes, que se comunican entre sí mediante canales y memoria compartida protegida:

- **Proceso de aplicación** (lector o escritor): ejecuta el bucle de trabajo del nodo (leer o escribir su copia del fichero) e invoca `PreProtocol`/`PostProtocol` para acotar cada acceso a la sección crítica.
- **Módulo `ra`** (Ricart-Agrawala generalizado): mantiene el estado del protocolo (relojes lógicos, permisos pendientes, postergaciones) y ejecuta la lógica de decisión.
- **Módulo `ms`** (sistema de mensajería, proporcionado): implementa la comunicación siguiendo las primitivas del *modelo actor*: `Send` asíncrono y directo (conexión TCP por mensaje, serialización con `gob`) y `Receive` bloqueante sobre un buzón (`mbox`).

2.2. Estructura interna de un nodo: tres flujos concurrentes

Siguiendo la estructura del algoritmo original en Algol visto en clase, cada nodo ejecuta tres flujos de ejecución (mediante gorutinas):

1. **Hilo principal (aplicación):** invoca el preprotocolo, accede a la sección crítica y ejecuta el postprotocolo.
2. **Gorutina de red (módulo `ms`):** acepta conexiones TCP entrantes, decodifica los mensajes y los deposita en el buzón `mbox`. Desacopla la recepción física del procesamiento lógico.
3. **Gorutina de protocolo (`recibirMensajes`):** extrae mensajes del buzón en bucle y ejecuta la lógica de Ricart-Agrawala: actualización del reloj lógico, evaluación de la matriz de exclusión y de la prioridad, envío o postergación de permisos, y conteo de los permisos recibidos.

Esta separación es imprescindible porque un nodo debe *atender simultáneamente* su propia voluntad de entrar en sección crítica y las peticiones ajenas, si el mismo hilo que espera permisos no pudiera procesar peticiones entrantes, el sistema se interbloquearía de inmediato (dos nodos esperándose mutuamente).

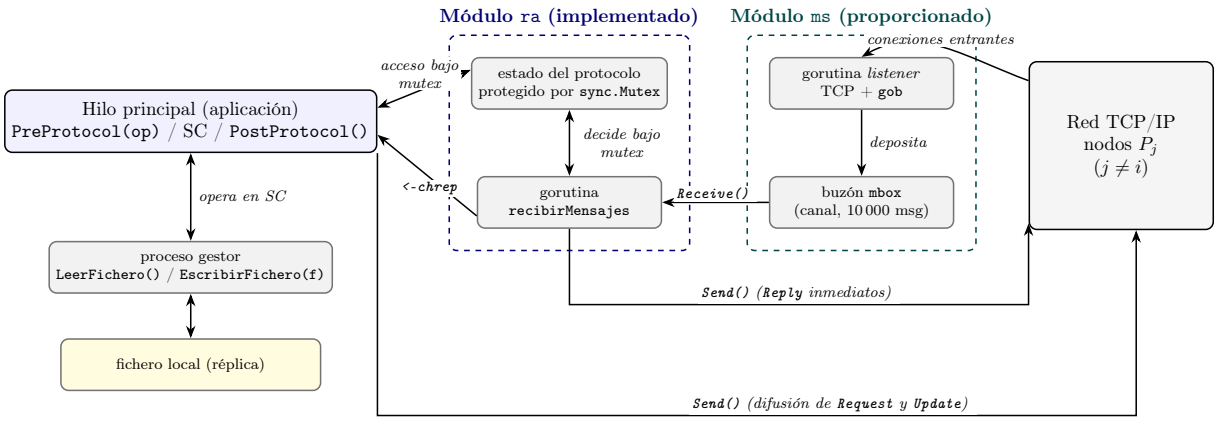


Figura 1: Arquitectura de componentes de un nodo. El hilo principal y la gorutina de protocolo comparten el estado del módulo `ra` bajo un único `sync.Mutex`, la gorutina de protocolo se sincroniza con el hilo principal mediante el canal `chrep` y con la red a través del buzón `mbox`. Todos los envíos de red de `Request` y `Update` desde el hilo principal y el envío de `Reply` desde la gorutina de protocolo se realizan siempre fuera de la región crítica local. (sección 4.6)

2.3. Gestor de ficheros y propagación de escrituras

Cada proceso posee su propia copia del fichero, gestionada por un *proceso gestor* con dos operaciones: `LeerFichero() string` (devuelve el contenido completo) y `EscribirFichero(fragmento string)` (añade un fragmento al final). Las lecturas se resuelven localmente dentro de la sección crítica de lectura. Las escrituras, además de aplicarse sobre la copia propia dentro de la sección crítica de escritura, se propagan al resto de nodos mediante un tercer tipo de mensaje, `Update`, registrado junto a `Request` y `Reply` en la inicialización del módulo `ms`.

La verificación de consistencia de mantener todos los nodos la misma copia se da por construcción, las escrituras se ejecutan en exclusión mutua global (sección 3.2), todas las copias aplican los fragmentos *en el mismo orden*, respetando el orden de concesión de la sección crítica. tras cada escritura, todos los ficheros contienen exactamente lo mismo. Para que ese orden sea realmente el mismo en todas las réplicas, los `Update` se emiten *dentro* de la sección crítica de

escritura, antes que cualquier permiso postergado del postprotocolo, así la aplicación de fragmentos en cada réplica respeta el orden causal de concesión de la sección crítica. Y la concurrencia de lecturas no afectan al documento por lo que no generan ningún problema.

3. Diseño del algoritmo de Ricart-Agrawala generalizado

3.1. De la versión clásica a la generalizada

El algoritmo clásico de Ricart-Agrawala resuelve el mutex distribuido con dos tipos de mensajes (REQUEST y REPLY) y un coste de $2(N-1)$ mensajes por entrada a sección crítica, sin requerir canales FIFO. La decisión de conceder un permiso se toma de forma inmediata al recibir cada petición, comparando relojes lógicos. La prioridad la da la tupla $(reloj, pid)$, gana la petición de reloj menor y, en caso de empate, la de *pid* menor, lo que proporciona un orden total y un resolución. La generalización a lectores y escritores introduce dos cambios sobre el esquema clásico:

1. Las peticiones llevan asociado el **tipo de operación** (`Read` o `Write`), que viaja en el mensaje `Request`.
2. La condición de postergación (instrucción (11) del algoritmo original) consulta la **matriz de exclusión**, donde una petición ajena solo compite con la propia si ambas operaciones son incompatibles.

3.2. Matriz de exclusión

Definimos una matriz booleana simétrica $exclude[op_1][op_2]$, con $exclude[op_1][op_2] = true$ si ambas operaciones no pueden ejecutarse simultáneamente:

	`Read`	`Write`
`Read`	`false` (compatibles)	`true` (conflicto)
`Write`	`true` (conflicto)	`true` (conflicto)

La regla de prioridad completa que evalúa un nodo i al recibir `Request`(k, j, op_j) es:

$$aplazar_i \;=\; ReqCS_i \;\wedge\; exclude[op_i][op_j] \;\wedge\; \big((seq_i < k) \vee (seq_i = k \wedge i < j)\big) \qquad (1)$$

Es decir: solo se posterga la respuesta si (a) el nodo propio también quiere entrar o está dentro de la sección crítica, (b) las dos operaciones se excluyen según la matriz, y (c) la petición propia tiene prioridad sobre la recibida. En cualquier otro caso, el permiso se envía de inmediato. La consecuencia directa es que dos lecturas concurrentes nunca se postergan entre sí, mientras que una escritura compite con todo.

3.3. Máquina de estados de un nodo

Respecto a la sección crítica, cada nodo transita por tres estados, equivalentes a los `cs_state` del algoritmo en Algol (`out`, `trying`, `in`). En nuestra implementación, los tres estados lógicos (`OUT`, `TRYING`, `IN`) se representan mediante la combinación de la variable `ReqCS` y el contador `OutRepCnt`. Los estados `TRYING` e `IN` comparten `ReqCS = true` porque la regla de postergación (evaluada por la gorutina de recepción) los trata de forma idéntica al decidir si retener un permiso ajeno. La transición real a la sección crítica ocurre cuando la condición `OutRepCnt == 0` libera el canal `chrep` que mantiene esperando al hilo principal.

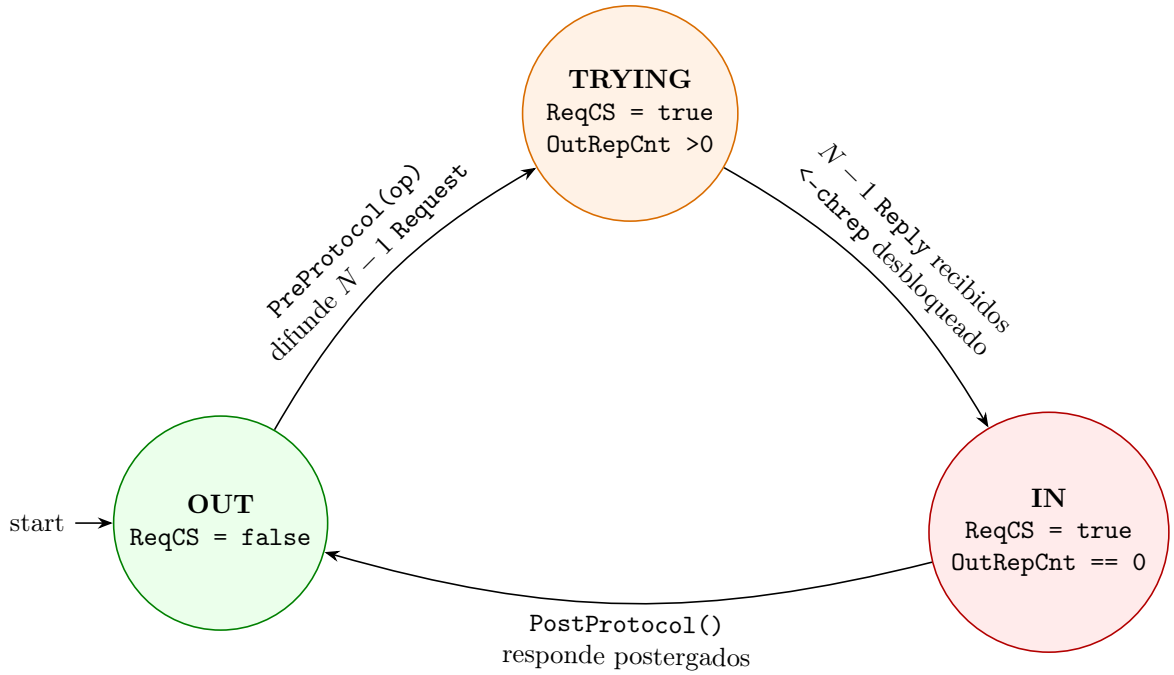


Figura 2: Máquina de estados mapeada a la implementación en Go. La transición entre estados se controla mediante la variable `ReqCS` (que engloba TRYING e IN para el cálculo de prioridad) y el contador de permisos `OutRepCnt`.

OUT. El nodo no desea acceder. Toda petición recibida se responde inmediatamente (la conjunción de la ecuación 1 es falsa porque $ReqCS_i = false$).

TRYING. El nodo ha fijado su reloj de petición, ha difundido `Request` a los $N-1$ nodos restantes y está suspendido hasta reunir los $N-1$ permisos. Las peticiones ajenas conflictivas de menor prioridad se postergan (`RepDefd`), las de mayor prioridad o no conflictivas se responden al instante.

IN. El nodo ejecuta su operación (lectura o escritura). Sigue atendiendo peticiones. Las conflictivas se postergan, las lecturas ajenas se admiten si la operación propia es también de lectura.

3.4. Modelo formal: red de Petri del comportamiento de un nodo

La máquina de estados anterior describe el ciclo de vida *propio* del nodo, pero no captura ni la difusión de las $N-1$ peticiones, ni la parte reactiva del protocolo (qué ocurre cuando llegan peticiones y permisos ajenos en cada estado), ni cómo se entregan después los permisos postergados. Para modelar el comportamiento completo usamos la red de Petri de la Figura 3, que combina en un solo grafo las tres zonas que ejecutan el hilo principal y la gorutina `recibirMensajes`, que es el ciclo de acceso a la sección crítica con su difusión de peticiones (parte izquierda), la atención a los mensajes recibidos (parte derecha) y el postprotocolo con el envío de los permisos retenidos (parte inferior).

La correspondencia entre los elementos de la red y la implementación es directa:

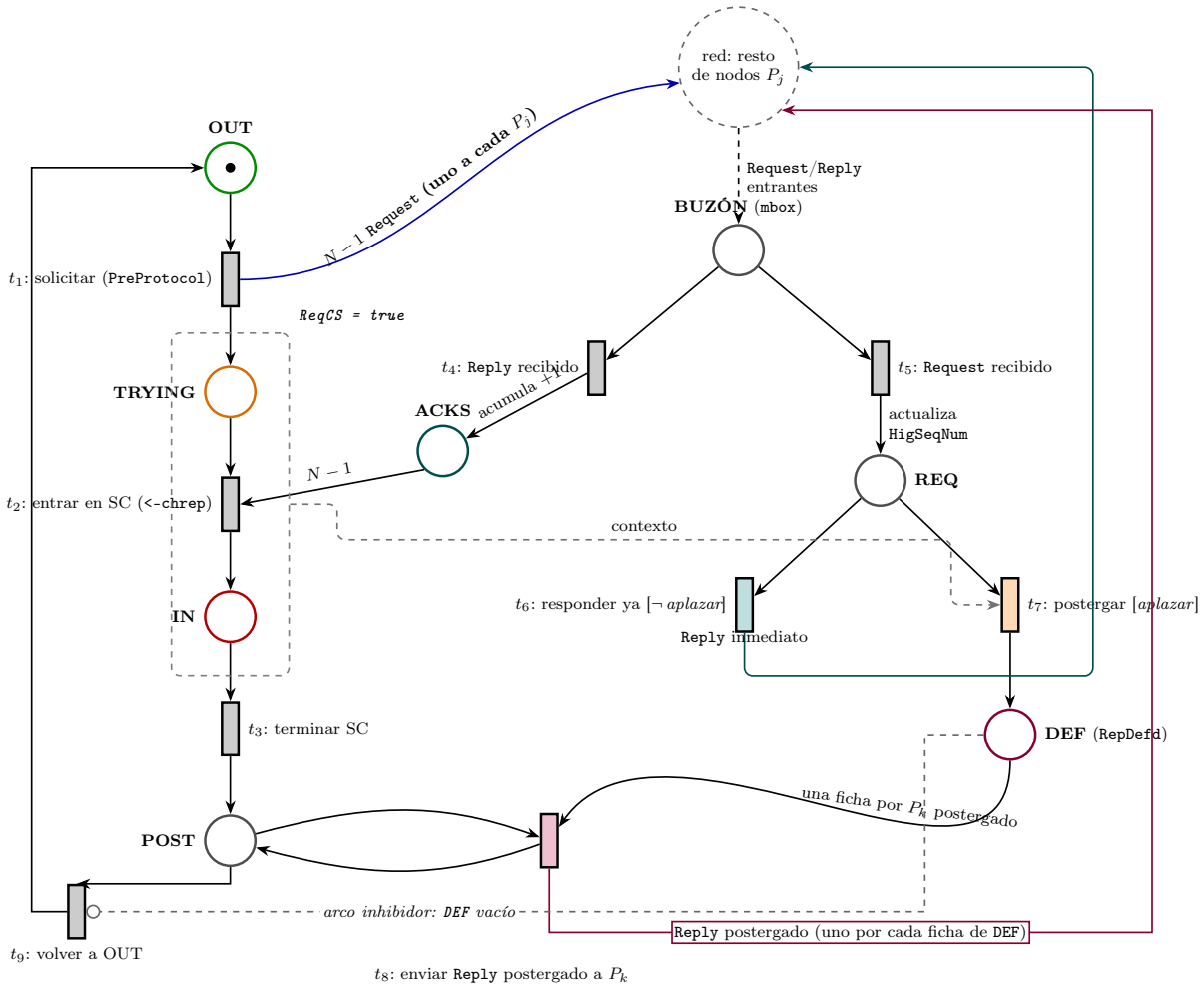


Figura 3: Red de Petri completa de un nodo. **Ciclo propio (izquierda):** t_1 modela el preprotocolo completo, donde además de llevar la ficha a `TRYING`, deposita las $N-1$ peticiones `Request` en la red (arco con peso $N-1$, uno por cada P_j, $j \neq i$). t_2 solo se dispara cuando `ACKS` reúne las $N-1$ respuestas (arco con peso $N-1$, condición `OutRepCnt == 0`). **Parte reactiva (derecha):** t_4 contabiliza cada `Reply` recibido, t_5 extrae cada `Request` del buzón y actualiza `HigSeqNum`, t_6 envía el permiso de inmediato (guarda $\neg$ *aplazar*) y t_7 lo posterga (guarda *aplazar*, ecuación 1), habilitado únicamente cuando hay ficha en la zona `ReqCS = true` (arco de contexto: comprueba la ficha sin consumirla). **Postprotocolo (abajo):** al terminar la SC la ficha pasa a `POST`, t_8 consume una ficha de `DEF` por cada proceso postergado, le envía su `Reply` retenido y devuelve la ficha a `POST` (bucle), cuando `DEF` queda vacío, t_9 devuelve el nodo a `OUT` (arco inhibidor: solo se dispara si `DEF` no tiene fichas). Marcado inicial: una ficha en `OUT`.

Elemento	Significado en el código
`OUT`, `TRYING`, `IN`	Los tres estados de la Figura 2 (`ReqCS`, `OutRepCnt`).
`POST`	El nodo ejecuta el postprotocolo: `ReqCS = false` y recorrido de `RepDefd` antes de volver a `OUT`.
`ACKS`	Permisos `Reply` contabilizados en `OutRepCnt`; el peso $N-1$ del arco hacia t_2 es la condición `OutRepCnt == 0`.
`BUZÓN`	Canal `mbox` del módulo `ms`; la red deposita mensajes sin orden garantizado.
`REQ`	Petición extraída del buzón, pendiente de evaluar por `handleRequest`.
`DEF`	Vector `RepDefd`: una ficha por cada proceso cuyo permiso se ha postergado.
t_1	`PreProtocol`: fija `OurSeqNum`, pone `ReqCS = true` y difunde las $N-1$ peticiones (arco de salida con peso $N-1$).
t_2, t_3	Concesión de la SC (`<-chrep`) y fin de la SC, ejecutados por el hilo principal.
t_4, t_5	Despacho por tipo en `recibirMensajes`: `handleReply` (decrementa `OutRepCnt`) y recepción de `Request` (actualiza `HigSeqNum`).
t_6, t_7	Las dos ramas de la ecuación 1: envío inmediato de `Reply` o marca `RepDefd[pid] = true`.
t_8	Cada iteración del bucle del `PostProtocol`: `Send(pid, Reply)` a un proceso postergado y borrado de su marca.
t_9	Fin del postprotocolo: retorno a `OUT` cuando no queda ningún permiso postergado (arco inhibidor).

La red permite razonar sobre todas las situaciones reales del sistema:

- **Acceso sin competencia:** t_1 deposita $N-1$ peticiones en la red y la ficha pasa a `TRYING`, donde la red devuelve $N-1$ respuestas que t_4 acumula en `ACKS`, t_2 las consume de golpe y el ciclo continúa `IN` $\rightarrow$ `POST` $\rightarrow$ `OUT`, con t_9 disparándose de inmediato porque `DEF` está vacío.
- **Petición ajena estando en OUT:** no hay ficha ni en `TRYING` ni en `IN`, luego el arco de contexto inhabilita t_7 y la petición solo puede salir por t_6 (respuesta inmediata).
- **Lector frente a lector:** hay ficha en `TRYING`/`IN`, pero la guarda de t_7 incluye $exclude$[`Read`][`Read`] = *false*, luego la petición también sale por t_6: ambas lecturas coexisten.
- **Conflicto con prioridad propia (p.ej. escritor):** ficha en `TRYING`/`IN`, operaciones excluyentes y tupla (*reloj*, *pid*) propia menor, se dispara t_7 y la ficha queda retenida en `DEF`. La red muestra explícitamente *cómo* se entrega después ese permiso, en el postprotocolo, t_8 consume la ficha de `DEF` y envía el `Reply` retenido a ese proceso, repitiéndose mientras queden fichas; solo cuando `DEF` queda vacío puede dispararse t_9 y cerrar el ciclo.
- **Conflicto con prioridad ajena:** la guarda de t_7 es falsa por la comparación de tuplas, luego se dispara t_6: el nodo cede el paso de inmediato, que es lo que deshace los empates sin comunicación extra.
- **Corrección estructural:** `OUT`, `TRYING`, `IN` y `POST` forman una componente conservativa (siempre exactamente una ficha: el nodo está en un único estado),`DEF` está acotado a $N-1$ fichas (cada proceso se posterga como mucho una vez por ronda) y `ACKS` a $N-1$, el arco inhibidor de t_9 garantiza que toda ficha depositada en `DEF` es consumida antes por t_8, lo que modela la *viveza*: ningún permiso postergado se pierde.

3.5. Diagramas de secuencia

En los tres diagramas siguientes se usa el mismo código de colores, donde las peticiones `Request` van en azul, los permisos `Reply` inmediatos en verde, los permisos *postergados* en naranja

discontinuo y las actualizaciones de réplica `Update` en morado punteado. Las barras sobre las líneas de vida marcan el tiempo en sección crítica (azul para lectura, rojo para escritura) y los círculos numerados ordenan la explicación.

3.5.1. Conflicto entre dos escritores (postergación y desempate)

La Figura 4 muestra el escenario crítico, dos escritores, P_1 y P_2, solicitan la sección crítica casi simultáneamente y sus relojes de petición *empatan* en 5. El desempate por *pid* resuelve el conflicto de forma unívoca en ambos nodos, sin necesidad de comunicación adicional.

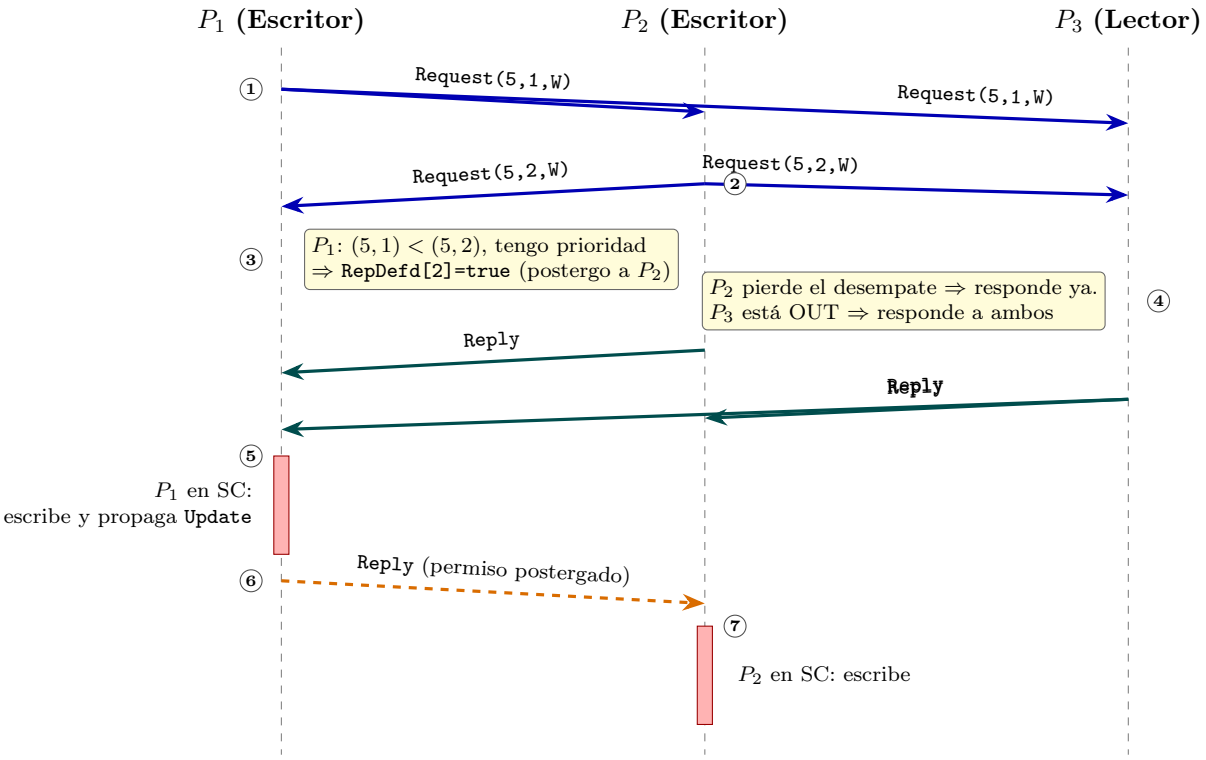


Figura 4: Dos escritores con relojes empatados. (1)–(2) P_1 y P_2 difunden sus peticiones, (3) P_1 compara $(5,1) < (5,2)$, comprueba que su petición tiene prioridad y *posterga* a P_2, (4) P_2 llega a la conclusión simétrica y responde de inmediato, igual que P_3, que está en OUT, (5) P_1 reúne los $N-1$ permisos y entra en SC, (6) al salir, `PostProtocol` envía el permiso postergado a P_2, (7) P_2 entra en SC.

Obsérvese que P_2 decide *de inmediato* al recibir la petición de P_1, sin esperar más mensajes. Esto es lo que permite al algoritmo funcionar sobre canales no ordenados (a diferencia del algoritmo de Lamport, que exige FIFO) y lo que reduce el coste a $2(N-1)$ mensajes por entrada.

3.5.2. Escritor frente a lector en sección crítica (propagación de la escritura)

La Figura 5 muestra el otro escenario conflictivo: un escritor P_2 pide acceso mientras el lector P_1 está leyendo. La escritura no puede adelantar a la lectura en curso (la matriz los marca como incompatibles y la petición de P_1 es más antigua), así que P_2 espera, cuando P_1 sale, recibe el permiso postergado, escribe y propaga el fragmento a todas las réplicas.

3.5.3. Lecturas concurrentes (sin postergación)

La Figura 6 muestra dos lectores que solicitan acceso simultáneamente. Al ser $exclude$[`Read`][`Read`] = `false`, la conjunción de la ecuación 1 es falsa en ambos nodos y los permisos se cruzan sin pos-

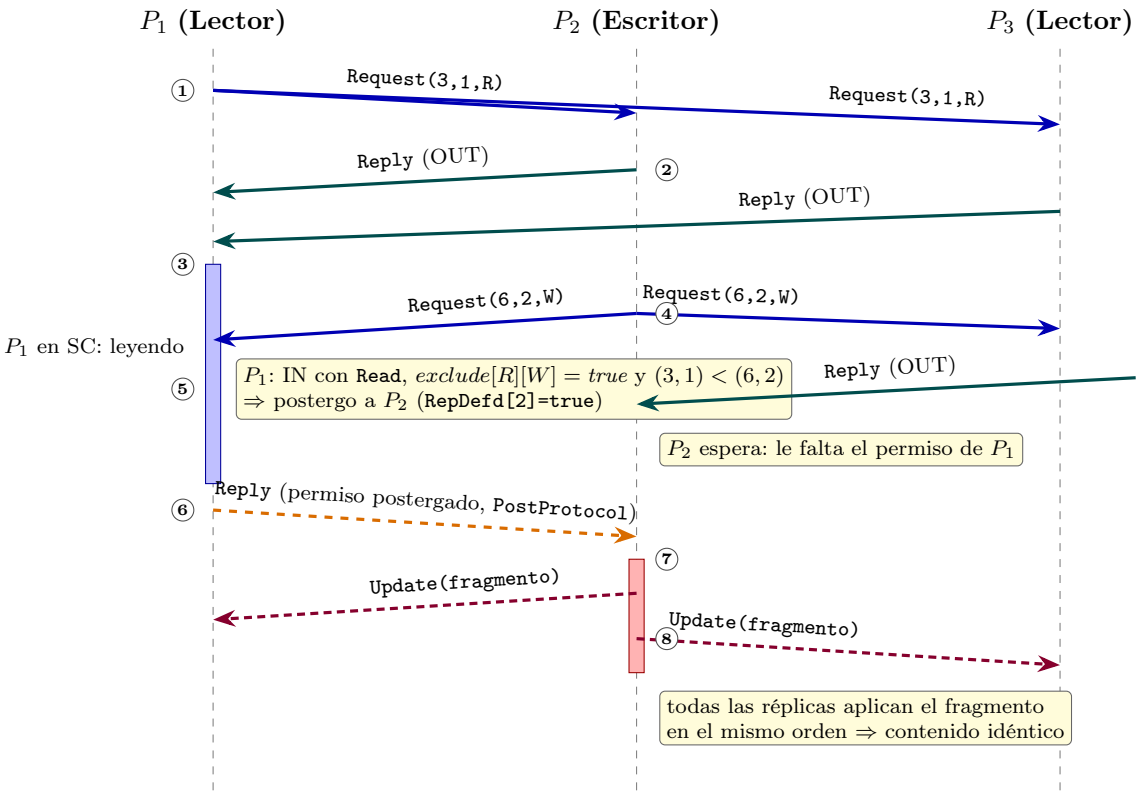


Figura 5: Lector en SC frente a escritor entrante. (1)–(2) P_1 obtiene los permisos sin competencia; (3) P_1 lee; (4) P_2 solicita la escritura, (5) P_1 lo posterga (conflicto y prioridad propia), mientras P_3 responde al estar en OUT, (6) al terminar la lectura, P_1 libera el permiso postergado, (7) P_2 entra en SC y escribe, (8) el fragmento se propaga a todas las réplicas con mensajes `Update`.

tergaciones, ambos lectores conviven en la sección crítica.

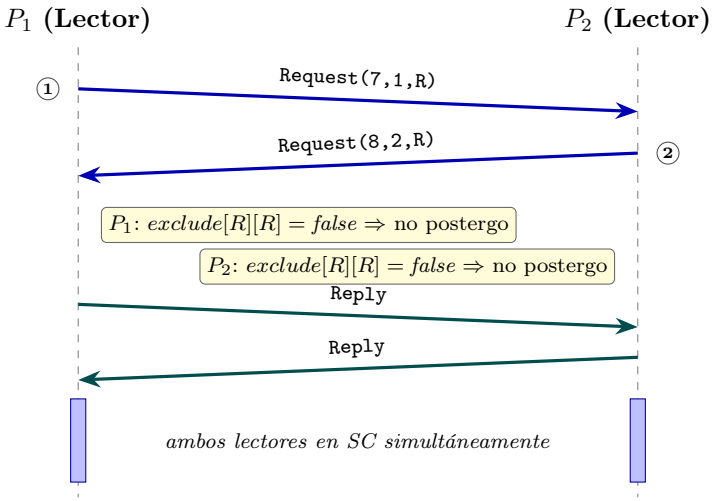


Figura 6: Dos lectores concurrentes: (1)–(2) ambos difunden su petición; al ser las operaciones compatibles, ninguno posterga al otro y los permisos se cruzan; ambos acceden a la vez. El coste sigue siendo $2(N-1)$ mensajes por entrada, pero el grado de concurrencia real es mayor que con un mutex puro.

4. Implementación en Go

Esta sección describe las decisiones de implementación relevantes del módulo `ra`, siguiendo la traducción del algoritmo en Algol (instrucciones (1)–(15)) a las primitivas de Go. Como advertía el guion, la Figura 1 del enunciado puede presentar condiciones de carrera, por ello la implementación se ha fundamentado en el código Algol original, prestando especial atención a la disciplina de mutex, es decir, todos los envíos de red se realizan siempre fuera de la sección crítica local. El motivo es evitar que el nodo se congele, si un hilo se pone a enviar un mensaje por la red mientras mantiene el mutex (sync.Mutex) cogido, bloquea el acceso al estado interno. Como la gorutina encargada de procesar los mensajes que llegan necesita ese mismo mutex constantemente, se quedaría atrapada esperando, el nodo dejaría de escuchar y se podría provocar un interbloqueo (deadlock) general en el sistema. Para solucionarlo, los datos se actualizan o clonan en un instante dentro del mutex, este se suelta de inmediato, y el envío por la red se hace de forma totalmente libre en el exterior.

4.1. Tipos de mensajes y serialización

Se definen dos tipos de mensaje de protocolo, registrados con `gob.Register` en el constructor para poder viajar por la interfaz genérica `ms.Message`:

```go
type Operation int

const (
    Read Operation = iota
    Write
)

var exclude = [2][2]bool{
    //  Read   Write
    {false, true}, // Read
    {true,  true}, // Write
}

type Request struct {
    //...
    Op    Operation  // tipo de operacion: entrada a la matriz
}

type Reply struct {
    Pid int // emisor del permiso (util para trazas y depuracion)
}
```

La extensión respecto al mutex clásico es el campo `Op`, permitiendo generalización, ya que permite al receptor evaluar $exclude[op_i][op_j]$ y acceder asi a la matriz y no a un mapa. El campo `Pid` de `Reply` no es estrictamente necesario para el conteo, pero sale rentable para las trazas de depuración (Sección **??**).

4.2. Estado del nodo

```
type RASharedDB struct {
    OurSeqNum   int  // reloj de mi peticion actual (lrd_i)
    HigSeqNum   int  // mayor reloj observado en la red
    OutRepCnt   int  // permisos que faltan para entrar en SC
    ReqCS       bool // true en TRYING e IN
    RepDefd     []bool // permisos postergados, indexado por pid (1..N)
    ms          *ms.MessageSystem
    done        chan bool
    chrep       chan bool // aviso: ya tengo los N-1 permisos
    Mutex       sync.Mutex // protege todo el estado anterior
    //Nuevo:
    me          int
    numProcesos int
    op          Operation // operacion de mi peticion actual
}
```

Tres decisiones de diseño de lo que nos han dado que merecen comentario:

- `RepDefd` **indexado por** *pid* (posiciones $1..N$, con la 0 sin usar): convierte la postergación y su liberación en accesos $O(1)$ y hace idempotente la marca de un mismo proceso.
- `chrep` **con buffer de tamaño 1**: la gorutina de protocolo *señaliza* la concesión de la sección crítica sin bloquearse, aunque el hilo principal aún no haya llegado a la lectura del canal. Con un canal sin buffer, la gorutina receptora quedaría retenida en el envío y dejaría de atender peticiones ajenas, con riesgo de interbloqueo.
- **Un único** `sync.Mutex` para todo el estado: la región crítica local es muy corta (comparaciones y asignaciones), por lo que un mutex único es la opción más simple y auditable; las operaciones de red se mantienen fuera de él (Sección 4.6).

  Adicionalmente a lo anterior, se han incorporado tres campos fundamentales al estado base proporcionado para completar la lógica distribuida:

  - **`me` y `numProcesos`:** Almacenan el identificador propio del nodo (*pid*) y el tamaño total del clúster (N). Son indispensables para resolver empates de prioridad, omitir el autoenvío de peticiones y saber exactamente cuántos permisos se deben esperar (inicializando `OutRepCnt` a $N-1$ en el preprotocolo).
  - **`op` (`Operation`):** Guarda el tipo de operación (`Read` o `Write`) que el nodo desea ejecutar o está ejecutando. Este campo es el núcleo de la generalización a lectores y escritores. Es obligatorio memorizar la operación propia durante el preprotocolo para que cuando la gorutina de recepción evalúe una petición ajena concurrente, pueda consultar la matriz de exclusión (`exclude[ra.op][r.Op]`) y decidir correctamente si debe postergarla o no.

4.3. Preprotocolo (instrucciones (1)–(6))

```
func (ra *RASharedDB) PreProtocol(op Operation) {
    ra.Mutex.Lock()
    ra.op        = op
    ra.ReqCS     = true                  // (1) cs_state <- trying
    ra.OurSeqNum = ra.HigSeqNum + 1      // (2) reloj de la peticion
    ra.OutRepCnt = ra.numProcesos - 1    // (3) waiting_from = R_i
    req := Request{Clock: ra.OurSeqNum, Pid: ra.me, Op: op}
    ra.Mutex.Unlock()

    for pid := 1; pid <= ra.numProcesos; pid++ { // (4) difusion
        if pid != ra.me {
            ra.ms.Send(pid, req)
        }
    }
    <-ra.chrep                           // (5) espera los N-1 permisos
}                                        // (6) cs_state <- in
```

Puntos críticos de esta traducción:

- **El reloj de la petición se calcula como** `HigSeqNum` $+ 1$ (instrucción (2) del Algol), no incrementando el valor anterior. Esto acopla el reloj propio al máximo observado en la red y evita que los relojes diverjan artificialmente. Además, hace que el reloj solo avance de verdad al *recibir* peticiones, lo que retrasa mucho su desbordamiento.
- **El reloj se fija una única vez**, dentro del mutex, y el mismo valor viaja en los $N-1$ mensajes. Incrementar el reloj dentro del bucle de envío (error frecuente en implementaciones ingenuas) haría que cada receptor viera una petición distinta, destruyendo el orden total y provocando evaluaciones asimétricas de prioridad que desencadenarían interbloqueos (*deadlocks*) irreversibles.
- **El envío se realiza fuera del mutex** sobre una copia local `req`, el estado puede seguir siendo atendido por la gorutina de protocolo mientras se difunde la petición.
- **La espera de permisos es una suspensión** (`<-ra.chrep`), no espera activa, el hilo principal queda dormido sin consumir CPU hasta que la gorutina de protocolo contabiliza el último `Reply`.

4.4. Recepción de mensajes (instrucciones (10)–(15))

La gorutina de protocolo despacha por tipo y ejecuta los manejadores:

```
func (ra *RASharedDB) handleRequest(r Request) {
    ra.Mutex.Lock()
    if r.Clock > ra.HigSeqNum {      // (10) clock_i <- max(clock_i, k)
        ra.HigSeqNum = r.Clock
    }
    miPrioridad := ra.OurSeqNum < r.Clock ||
        (ra.OurSeqNum == r.Clock && ra.me < r.Pid)
    aplazar := ra.ReqCS && exclude[ra.op][r.Op] && miPrioridad // (11)
    if aplazar {
        ra.RepDefd[r.Pid] = true     // (12) perm_delayed <- perm_delayed U {j}
    }
    ra.Mutex.Unlock()

    if !aplazar {
        ra.ms.Send(r.Pid, Reply{Pid: ra.me}) // (13) permiso inmediato
    }
}
```

```
18
19
20 func (ra *RASharedDB) handleReply(rep Reply) {
21     ra.Mutex.Lock()
22     ra.OutRepCnt--                    // (15) waiting_from <- waiting_from \ {j}
23     todos := ra.OutRepCnt == 0
24     ra.Mutex.Unlock()
25     if todos {
26         ra.chrep <- true              // desbloquea (5) del preprotocolo
27     }
28 }
```

La evaluación completa de la ecuación 1 actualización del máximo, comparación de tuplas $(reloj, pid)$ y marca de postergación se ejecuta de forma *atómica* bajo el mutex. Es la única forma de que la decisión sea coherente, si la comparación y la marca se hicieran en dos pasos, dos peticiones cruzadas podrían observar estados intermedios inconsistentes (por ejemplo, ambos nodos postergándose mutuamente, es decir interbloqueo). El envío del `Reply`, de nuevo, se hace fuera del mutex.

Conforme a la teoría, el reloj lógico solo se actualiza al recibir `Request` (instrucción (10)), la recepción de `Reply` no altera los relojes, lo que contribuye a que su valor crezca lentamente.

4.5. Postprotocolo (instrucciones (7)–(9))

```
1  func (ra *RASharedDB) PostProtocol() {
2      var pendientes []int
3      ra.Mutex.Lock()
4      ra.ReqCS = false                      // (7) cs_state <- out
5      for pid := 1; pid <= ra.numProcesos; pid++ {
6          if ra.RepDefd[pid] {              // (8) para cada postergado
7              ra.RepDefd[pid] = false       // (9) perm_delayed <- {}
8              pendientes = append(pendientes, pid)
9          }
10     }
11     ra.Mutex.Unlock()
12
13     for _, pid := range pendientes {
14         ra.ms.Send(pid, Reply{Pid: ra.me}) // envio fuera del mutex
15     }
16 }
```

La lectura y el borrado del vector `RepDefd` se hacen *dentro* del mutex (comparte estado con `handleRequest`), pero los envíos se realizan *fuera*, sobre la lista `pendientes` previamente copiada. La accion de *copiar bajo mutex, comunicar sin mutex* es esencial por dos razones:

1. `ms.Send` realiza un `net.Dial` bloqueante por mensaje, retener el mutex durante la operación de red congelaría a la gorutina de protocolo (que necesita el mismo mutex para atender peticiones), degradando o bloqueando el nodo.

2. La semántica del algoritmo no se ve alterada, una vez `ReqCS` = *false*, cualquier petición nueva se responde de inmediato, que es exactamente lo que debe ocurrir cuando el nodo está en OUT.

4.6. Condiciones de carrera consideradas

Además de la atomicidad de la decisión de postergación ya comentada, la implementación cierra las siguientes carreras, presentes de forma latente en el esquema de la Figura 1 del guion:

- **Reloj de petición vs. recepción concurrente:** fijar `OurSeqNum` y marcar `ReqCS` en la misma región crítica impide que una petición ajena sea evaluada contra un estado a medio configurar (p.ej. `ReqCS` ya a *true* pero `op` sin asignar).
- **Carrera sobre `RepDefd` entre `handleRequest` y `PostProtocol`:** ambos acceden al vector; al ejecutarse ambas secciones bajo el mismo mutex, un permiso no puede ni postergarse dos veces ni perderse en la transición OUT.
- **Conteo de permisos:** `OutRepCnt` solo se decrementa bajo mutex y la señal a `chrep` se emite tras liberarlo, evitando tanto dobles señales como retenciones del canal con el mutex tomado.
- **Pérdida del despertar:** el buffer de `chrep` garantiza que la señal no se pierde aunque el hilo principal aún no esté bloqueado en la recepción (ventana entre el último `Send` del preprotocolo y `<-chrep`).

5. Pruebas y validación

5.1. Estrategia

5.2. Batería de pruebas

5.3. Criterios de corrección comprobados

6. Limitaciones conocidas y trabajo futuro

- **Inanición potencial de escritores.** Con la matriz básica, un escritor postergado puede ser adelantado indefinidamente por lectores nuevos (que nunca se postergan entre sí). Es una propiedad inherente a la generalización simple del algoritmo, no un defecto de la implementación, una extensión natural sería envejecer las peticiones de escritura o cerrar la admisión de nuevos lectores mientras haya un escritor postergado.
- **Tolerancia a fallos.** Como todo el tema, el algoritmo asume ausencia de fallos de proceso y de comunicación: la caída de un nodo con peticiones pendientes deja al sistema bloqueado. Una extensión con *timeouts* y retransmisiones queda fuera del alcance de la práctica.

7. Conclusión